

Python Collections

Container Data Types

- Container data types store and organize multiple values.
- Allow efficient data management and retrieval.
- Can hold different types of objects.
- Form the foundation of most Python programs.
- Include Lists, Tuples, Sets, and Dictionaries.

```
numbers = [1, 2, 3]
```

```
person = ("John", 25)
```

```
colors = {"red", "blue"}
```

```
student = {"name": "Alice"}
```

Lists

- Lists are ordered and mutable collections.
- Elements are accessed using indexes.
- Allow duplicate values.
- Support adding, removing, and updating items.
- Created using square brackets [].

```
fruits = ["apple", "banana", "orange"]
```

```
fruits.append("grape")
```

```
print(fruits)
```

List Operations

- `append()` adds an item to the end.
- `remove()` deletes a specific item.
- `insert()` adds an item at a specific position.

- `pop()` removes an item by index.
- `sort()` arranges items in order.

```
numbers = [3, 1, 4]
```

```
numbers.sort()
```

```
print(numbers)
```

List Comprehensions

- Create lists using concise syntax.
- Faster and more readable than loops.
- Useful for filtering and transformations.
- Reduce boilerplate code.
- Common in data processing.

```
squares = [x * x for x in range(5)]
```

```
print(squares)
```

Tuples

- Tuples are ordered collections.
- Immutable after creation.
- Allow duplicate values.
- Faster and more memory-efficient than lists.
- Created using parentheses `()`.

```
person = ("John", 25, "Developer")
```

```
print(person[0])
```

Tuple Features

- Cannot be modified after creation.
- Useful for fixed data.
- Support indexing and slicing.

- Can be unpacked into variables.
- Often used for returning multiple values.

```
person = ("John", 25)
```

```
name, age = person
```

```
print(name)
```

```
print(age)
```

Sets

- Sets store unique values only.
- Unordered collections.
- Automatically remove duplicates.
- Support mathematical set operations.
- Created using curly braces {}.

```
numbers = {1, 2, 3, 3, 2}
```

```
print(numbers)
```

Set Operations

- `add()` inserts new values.
- `remove()` deletes values.
- `union()` combines sets.
- `intersection()` finds common values.
- `difference()` finds unique values.

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
print(set1.intersection(set2))
```

Dictionaries

- Store data as key-value pairs.

- Keys must be unique.
- Values can be any data type.
- Provide fast lookups.
- Created using curly braces {}.

```
student = {"name": "Alice", "age": 20}
```

```
print(student["name"])
```

Dictionary Operations

- Access values using keys.
- Add new entries dynamically.
- Update existing values easily.
- Remove entries with pop().
- Iterate through keys and values.

```
student = {"name": "Alice"}
```

```
student["age"] = 20
```

```
print(student)
```

Comparing Container Types

- Lists are ordered and mutable.
- Tuples are ordered and immutable.
- Sets are unordered and unique.
- Dictionaries store key-value pairs.
- Each container serves different use cases.

```
data_list = [1, 2, 3]
```

```
data_tuple = (1, 2, 3)
```

```
data_set = {1, 2, 3}
```

```
data_dict = {"a": 1}
```

Choosing the Right Container

- Use lists for dynamic ordered data.
- Use tuples for fixed data.
- Use sets for uniqueness checks.
- Use dictionaries for mappings and lookups.
- Selection impacts readability and performance.

Customizing Built-In Data Types

- Python allows extending container behavior.
- collections module provides helper classes.
- Custom containers inherit built-in functionality.
- Additional validation and features can be added.
- Useful for domain-specific data structures.

```
from collections import UserList
```

UserList

- Wrapper around the built-in list type.
- Easier to customize than inheriting list directly.
- Supports overriding methods.
- Useful for adding validation.
- Behaves like a normal list.

```
from collections import UserList
```

```
class PositiveNumbers(UserList):
```

```
    def append(self, item):
```

```
        if item > 0:
```

```
            super().append(item)
```

```
nums = PositiveNumbers()
```

```
nums.append(5)
```

```
print(nums)
```

UserDict

- Wrapper around the built-in dictionary.
- Simplifies dictionary customization.
- Allows custom key and value validation.
- Supports method overriding.
- Behaves like a standard dictionary.

```
from collections import UserDict
```

```
class UpperDict(UserDict):
```

```
    def __setitem__(self, key, value):
```

```
        super().__setitem__(key.upper(), value)
```

```
data = UpperDict()
```

```
data["name"] = "John"
```

```
print(data)
```

UserString

- Wrapper around the built-in string type.
- Allows custom string behavior.
- Useful for text validation.
- Supports method extension.
- Behaves like a regular string.

```
from collections import UserString
```

```
class ShoutString(UserString):
```

```
    def shout(self):
```

```
        return self.data.upper()
```

```
text = ShoutString("hello")
```

```
print(text.shout())
```

Benefits of Custom Containers

- Promote code reuse.
- Encapsulate validation logic.
- Improve maintainability.
- Extend built-in behavior safely.
- Reduce duplicate code.

```
from collections import UserList
```

```
class EvenList(UserList):
```

```
    def append(self, item):
```

```
        if item % 2 == 0:
```

```
            super().append(item)
```